NAAN MUDHALVAN – PROJECT REPORT

SERVICE NOW PRACTICAL PROJECT

IMPLEMENT CLIENT SCRIPT & UI POLICY (INCIDENT)

A Project Report Submitted as Part of the Naan Mudhalvan Training

Submitted by

Name & Register Number

P. Valarmathi, C4S47515

V. Nallammal, C4S47514

V. Murugeshwari, C4S47512

M. Musbera, C4S47513

M. Vettaikaruppan, C4S47536

Department: B.Sc. Computer Science

College: Government Arts and Science College Thiruchuli

Academic Year: 2026–2027

Naan Mudhalvan – ServiceNow / IT Service Management

CERTIFICATE

This is to certify that the project report entitled “Implement Client Script & UI Policy (Incident)” is a bonafide work carried out by _____ of _____ during the academic year 2026–2027 as part of the Naan Mudhalvan training/project work.

The work focuses on understanding and implementing client-side form behaviour in ServiceNow Incident Management using Client Scripts and UI Policies. The project demonstrates conditional field behaviour, validation, mandatory fields, visibility, and read-only controls on Incident form.

The project work has been completed under the guidance of the concerned faculty/trainer and is submitted for academic evaluation.

Faculty / Trainer Signature

_____

Head of Department: B.Sc Computer Science

_____

DECLARATION

I hereby declare that the project report entitled “Implement Client Script & UI Policy (Incident)” is prepared by me as part of the Naan Mudhalvan training/project activity. The report presents the concepts, design, implementation approach, testing procedure, and expected results of the ServiceNow Incident form exercise.

I have used the learning resources and documentation required to understand ServiceNow Client Scripts and UI Policies. Wherever external technical documentation has been consulted, it is acknowledged in the References section.

Place: _____

Date: _____

Signature: _____

ACKNOWLEDGEMENT

I sincerely thank my college management and faculty members for providing the opportunity to participate in the Naan Mudhalvan training programme. I am grateful to the trainers and mentors who provided guidance on ServiceNow, Incident Management, client-side scripting, and user interface configuration.

I also thank my department for providing the required academic support to complete this project report. Finally, I thank everyone who supported me directly or indirectly during the preparation, testing, and documentation of this project.

TABLE OF CONTENTS

1. Brainstorming & Ideation Phase

2. Requirement Analysis Phase

3. Project Design Phase

4. Project Planning Phase

5. Project Development Phase

6. Project Testing Phase

7. Project Documentation Phase

8. Project Demonstration Phase

Conclusion

References

1. BRAINSTORMING & IDEATION PHASE

1.1 Project Overview

The project “Implement Client Script & UI Policy (Incident)” is designed around client-side behaviour on the ServiceNow Incident form. The main idea is to make the Incident form respond to user actions in a controlled and understandable manner. Instead of allowing every field to behave in the same way, the project introduces conditional behaviour such as mandatory fields, visibility changes, read-only controls, and validation.

1.2 Problem Identification

An Incident form contains information required to record and manage an issue. If fields are not controlled properly, users may leave important information empty, enter information in an unsuitable situation, or see fields that are not relevant to the current incident. The project addresses these form-level usability requirements through Client Scripts and UI Policies.

1.3 Proposed Idea

The proposed solution is to configure client-side rules on the Incident form. Client Scripts are used when scripting is required for client-side behaviour or validation. UI Policies are used for declarative form behaviour such as making a field mandatory, visible, or read-only based on conditions.

1.4 Objectives

- Understand Incident form behaviour.
- Apply client-side validation and interaction.
- Control field visibility and mandatory state.
- Apply read-only behaviour where required.
- Test the behaviour under different field conditions.
- Document the implementation in a phase-wise format.

1.5 Expected Outcome

The expected outcome is an Incident form in which relevant fields respond to the selected conditions. The user receives a clearer form experience, important information can be enforced where necessary, and unnecessary editing can be restricted through client-side controls.

2. REQUIREMENT ANALYSIS PHASE

2.1 Functional Requirements

- The Incident form shall allow users to enter and update incident information.
- Conditional rules shall control selected fields according to the form state.
- Required fields shall be made mandatory when the defined condition is satisfied.
- Fields that are not relevant to a condition shall be hidden where required.
- Fields that should not be changed under a condition shall become read-only.
- Client-side validation shall prevent unsuitable or incomplete input where defined.
- Each rule shall be tested with both matching and non-matching conditions.

2.2 Non-Functional Requirements

The configuration should be understandable, maintainable, and consistent with the Incident form. The rules should respond without unnecessary navigation and should provide predictable behaviour to the user. Naming of scripts and policies should clearly communicate their purpose.

2.3 User Requirements

The user needs a form that guides data entry. Important information should be requested at the appropriate time, while fields that are not applicable should not distract the user. The user should be able to understand why a field is required, visible, or restricted through the configured form behaviour.

2.4 Technical Requirements

- ServiceNow instance with access to Incident Management.
- Incident table and form.
- Client Script configuration.
- UI Policy configuration.
- A browser for accessing the ServiceNow interface.
- Test records or controlled test conditions.

2.5 Requirement Traceability

The project connects each requirement with an implementation mechanism. Conditional validation is associated with Client Scripts, while declarative visibility, mandatory state, and read-only behaviour are associated with UI Policies. Testing then verifies whether each requirement produces the expected form behaviour.

3. PROJECT DESIGN PHASE

3.1 Design Approach

The design separates the project into form conditions, client-side scripting, UI Policy rules, and testing. This separation makes it easier to understand which configuration is responsible for each behaviour.

3.2 Incident Form Design

The Incident form acts as the main user interface. The selected incident information becomes the input to the configured conditions. When the condition changes, the client-side rules determine how selected fields should behave.

3.3 Client Script Design

A Client Script is designed for situations where JavaScript-based client-side logic is needed. The script receives the current form state and can validate or modify form behaviour according to the project requirement. The script should be limited to the required client-side task and should use clear field references.

3.4 UI Policy Design

A UI Policy is designed around a condition and the resulting form actions. When the condition is true, the configured field actions are applied. When the condition is false, the form can return to the defined opposite state depending on the configuration.

3.5 Logical Flow

User opens Incident form → user enters or changes incident information → condition is evaluated → Client Script/UI Policy responds → selected fields become visible, hidden, mandatory, or read-only → user continues data entry → result is verified.

3.6 Design Principles

- Keep rules focused on one clear purpose.
- Use meaningful names for scripts and policies.
- Avoid unnecessary client-side logic.
- Keep conditional behaviour consistent.
- Test both condition true and condition false states.

4. PROJECT PLANNING PHASE

4.1 Work Breakdown

Phase 1: Understand the project idea and identify the form problem.

Phase 2: List functional, non-functional, user, and technical requirements.

Phase 3: Design the Client Script, UI Policy, fields, conditions, and flow.

Phase 4: Prepare implementation and testing sequence.

Phase 5: Configure and implement the rules in ServiceNow.

Phase 6: Execute test cases and record expected/actual behaviour.

Phase 7: Prepare report, screenshots, configuration details, and references.

Phase 8: Demonstrate the completed form behaviour.

4.2 Implementation Schedule

The implementation is planned in small steps so that each configuration can be tested before moving to the next. First, the Incident form and required fields are identified. Next, the UI Policies and Client Scripts are configured. After each rule is created, the form is tested using relevant conditions. Finally, the complete flow is demonstrated and documented.

4.3 Risk Considerations

Possible implementation issues include incorrect field names, conditions that do not match the intended form state, conflicting UI Policies, script errors, and testing only one condition. These risks are reduced by using clear names, testing both states, and checking browser console or ServiceNow feedback when necessary.

5. PROJECT DEVELOPMENT PHASE

5.1 Development Environment

The project is implemented in a ServiceNow environment using the Incident table. Client-side configuration is performed through Client Scripts and UI Policies. The browser is used to access the ServiceNow interface and verify the resulting form behaviour.

5.2 Client Script Implementation

The Client Script is used to implement client-side logic required by the project. Depending on the requirement, the script can execute when a form loads, when a field changes, or before a client-side submission. The implementation should use the appropriate trigger so that the rule runs at the correct time.

5.3 Client-Side Validation

Validation checks whether the information entered by the user satisfies the defined requirement. If the condition is not satisfied, the user is informed and the form is prevented from proceeding where appropriate. The purpose is to improve data quality at the point of entry.

5.4 UI Policy Implementation

UI Policies provide condition-based form behaviour without requiring every field-state change to be written manually in JavaScript. A policy can control whether a field is mandatory, visible, or read-only according to the selected condition.

5.5 Conditional Behaviour

For the Incident form, a condition may be based on an incident field or form state. When the condition becomes true, the associated field action is applied. When the condition becomes false, the opposite or default state is restored according to the configuration.

5.6 Implementation Sequence

1. Open the Incident configuration area.
2. Identify the required Incident fields.
3. Create the required UI Policy conditions.
4. Configure mandatory, visible, or read-only actions.
5. Create the Client Script for validation or dynamic logic.
6. Save the configuration.
7. Open an Incident record and test the behaviour.
8. Modify the condition and verify the alternate state.

5.7 Implementation Result

The completed configuration provides a controlled Incident form where the selected client-side rules respond to user actions. The implementation demonstrates the practical use of ServiceNow Client Scripts and UI Policies for form-level interaction.

6. PROJECT TESTING PHASE

6.1 Testing Objective

Testing verifies that each Client Script and UI Policy behaves according to the requirement. The test process checks both the condition where the rule should apply and the condition where it should not apply.

6.2 Test Cases

TC01 – Open Incident form
Expected: Form loads normally
Actual/Status: Form loads without unwanted behaviour / Pass/To be recorded

TC02 – Condition is false
Expected: Controlled fields remain in default state
Actual/Status: Default state is displayed / Pass/To be recorded

TC03 – Condition becomes true
Expected: Configured field behaviour changes
Actual/Status: Field becomes mandatory/visible/read-only as designed / Pass/To be recorded

TC04 – Condition returns false
Expected: Previous conditional state is removed
Actual/Status: Field returns to defined opposite/default state / Pass/To be recorded

TC05 – Invalid input
Expected: Validation responds
Actual/Status: User is informed and invalid action is restricted / Pass/To be recorded

TC06 – Valid input
Expected: Form accepts valid data
Actual/Status: No validation error is shown / Pass/To be recorded

TC07 – Repeated field changes
Expected: Rules remain stable
Actual/Status: No unexpected behaviour occurs / Pass/To be recorded

6.3 Testing Result

Testing should be completed in the actual ServiceNow instance before final submission. The final report should use the real observed result and screenshots from the user's own configuration. The test table above provides the structure for recording those results.

6.4 Defect Handling

If a rule does not behave as expected, the condition, field name, script trigger, UI Policy action, and possible conflicts should be checked. After correction, the same test case should be executed again to confirm the change.

7. PROJECT DOCUMENTATION PHASE

7.1 Purpose of Documentation

Documentation records the project objective, requirements, design, implementation, testing, and final behaviour. It also makes the configuration easier to understand and reproduce.

7.2 Configuration Documentation

Each Client Script should be documented with its name, table, trigger type, condition or purpose, and the behaviour it produces. Each UI Policy should be documented with its name, table, condition, and field actions.

7.3 Client Script Documentation Format

Script Name: _____

Table: Incident

Type: _____

Condition/Purpose: _____

Functionality: _____

Expected Result: _____

7.4 UI Policy Documentation Format

Policy Name: _____

Table: Incident

Condition: _____

Field Action: Mandatory / Visible / Read-only

Expected Result: _____

7.5 Screenshot Documentation

Screenshots should be taken from the completed ServiceNow configuration and Incident form. Suitable screenshots include the Client Script configuration, UI Policy configuration, Incident form before the condition, Incident form after the condition, validation result, and final successful behaviour.

7.6 References

ServiceNow learning material and documentation used during the Naan Mudhalvan ServiceNow training should be listed here. Only resources actually consulted for the implementation should be included.

8. PROJECT DEMONSTRATION PHASE

8.1 Demonstration Objective

The demonstration presents the completed Incident form behaviour from start to finish. It should show the project name, the relevant ServiceNow configuration, the Incident form, the condition being applied, and the resulting field behaviour.

8.2 Demonstration Flow

9. Introduce the project title: Implement Client Script & UI Policy (Incident).
10. Open the ServiceNow Incident form.
11. Show the relevant Client Script/UI Policy configuration.
12. Demonstrate the initial form state.
13. Change the field or condition used by the rule.
14. Show the resulting mandatory, visible, read-only, or validation behaviour.
15. Demonstrate the alternate condition where applicable.
16. Summarize the final result.

8.3 Demonstration Evidence

The final evidence should be based on the actual project instance. Screenshots and the demonstration recording should show the implemented configuration and its visible result. The project name should be clearly stated during the demonstration.

8.4 Final Outcome

The project demonstrates practical understanding of ServiceNow Incident form configuration using Client Scripts and UI Policies. The completed work shows how client-side rules can improve controlled data entry and user interaction on an Incident form.

CONCLUSION

The “Implement Client Script & UI Policy (Incident)” project provides a practical implementation of client-side form behaviour in ServiceNow. The project was organized through eight phases beginning with brainstorming and requirement analysis, followed by design and planning, development, testing, documentation, and demonstration. The implementation focuses on conditional behaviour, validation, mandatory fields, visibility, and read-only controls on the Incident form.

The project also provides an organized method for checking the configuration before final submission. Actual ServiceNow screenshots, observed test results, and demonstration evidence should be inserted after the implementation is completed in the user's instance.

REFERENCES

- ServiceNow learning resources provided as part of Naan Mudhalvan ServiceNow training.
- ServiceNow platform documentation consulted during implementation.
- Project-specific training/reference materials provided during the project development sessions.